Università degli Studi di Verona

CORSO DI LAUREA IN INFORMATICA

Algoritmi

Federico Brutti
federico.brutti@studenti.univr.it

Inserire citazione inerente alla materia

Indice

1 Fondamenti **4**
1.1 Complessità e notazione asintotica 4
1.2 Equazioni di ricorrenza 8
1.3 Teorema dell'esperto 11
1.4 Esercizi svolti 13

2 Ordinamenti e selezioni **14**
2.1 Problema dell'ordinamento 14
2.2 Insertion Sort 14
2.3 Merge Sort 15
2.4 Struttura Heap e Heap Sort 17
2.5 Quick Sort classico e probabilistico 19
2.6 Counting Sort 19
2.7 Radix Sort 19
2.8 Bucket Sort 19
2.9 Problema della selezione 19

3 Strutture dati **20**
3.1 Heap 20
3.2 Alberi binari 20
3.2.1 RB-alberi 20
3.2.2 B-alberi binomiali 20
3.3 Tabelle hash 20
3.4 Code con priorità 20
3.5 Insiemi disgiunti 20
3.6 Estensione di strutture dati 20
3.7 Grafi 20

4 Progetto e analisi di algoritmi **21**
4.1 Divide et impera 21
4.2 Programmazione greedy 21

4.3 Programmazione dinamica 21
4.4 Ricerca locale 21
4.5 Backtracking 21
4.6 Branch and bound 21

5 Algoritmi fondamentali 22
5.1 Alberi di copertura di costo minimo 22
5.2 Programmazione lineare 22
5.3 Cammini minimi 22
5.3.1 Sorgente singola 22
5.3.2 Sorgente multipla 22
5.4 Flusso massimo 22
5.5 Matching massimale su grafo bipartito 22

Capitolo 1

Fondamenti

Il corso di Algoritmi ha l'obiettivo di fornire gli strumenti teorici e metodologici per la progettazione, descrizione e analisi formale di algoritmi. In particolare, studieremo come specificare in modo rigoroso una procedura di calcolo, come dimostrarne la correttezza e valutarne l'efficienza.

Definiamo **algoritmo** come una procedura di calcolo ben definita che prende uno o più valori in input per generarne ulteriori in output con un numero finito di passi. Di ogni algoritmo analizzeremo la **complessità**, intesa come funzione che misura le risorse necessarie alla sua esecuzione, intese tipicamente come tempo e spazio, in relazione alla dimensione dell'input. Questo ci permetterà di confrontare soluzioni alternative e scegliere quella più efficiente rispetto al problema considerato.

1.1 Complessità e notazione asintotica

La **complessità** di un algoritmo è una funzione che descrive la quantità di risorse necessarie alla sua esecuzione in relazione alla dimensione dell'input. Tipicamente la consideriamo in termini di **complessità temporale**, che riguarda il tempo di esecuzione, e di **complessità spaziale**, per la memoria utilizzata.

Non si tratta di un valore fisso, ma una funzione $T(n)$, dove n rappresenta la dimensione dell'input. Al variare di n, varia anche il numero di operazioni eseguite dall'algoritmo. Poiché siamo interessati al comportamento dell'algoritmo per input di grandi dimensioni, utilizziamo l'**analisi asintotica**, che permette di descrivere l'ordine di crescita della funzione trascurando costanti moltiplicative e termini di ordine inferiore. In particolare presenta le seguenti notazioni:

- **Limite asintotico superiore**: $O(f(n))$

Date due funzioni $f, g : \mathbb{N} \to \mathbb{R}_{\geq 0}$, diciamo che $f(n) \in O(g(n))$ se esistono una costante c e un valore $\overline{n}$ entrambi positivi tali che:

$$0 \leq f(n) \leq cg(n) \quad \forall n \geq \overline{n}$$

In tal caso g fornisce un limite superiore asintotico per f. Intuitivamente, per un n sufficientemente grande, la funzione f cresce al più come g, a meno di una costante moltiplicativa. Formalmente:

$$f(n) \in O(g(n)) \iff \exists c > 0, \exists \overline{n} : \forall n \geq \overline{n}, f(n) \leq cg(n)$$

- **Limite asintotico inferiore**: $\Omega(f(n))$

  Date due funzioni $f, g : \mathbb{N} \to \mathbb{R}_{\geq 0}$, diciamo che $f(n) \in \Omega(g(n))$ se esistono una costante c e un valore $\overline{n}$ entrambi positivi tali che:

  $$0 \leq cg(n) \leq f(n) \quad \forall n \geq \overline{n}$$

  In questo caso g fornisce un limite inferiore asintotico per f. Formalmente:

  $$f(n) \in \Omega(g(n)) \iff \exists c > 0, \exists \overline{n} : \forall n \geq \overline{n}, f(n) \geq cg(n)$$

- **Limite asintotico stretto**: $\Theta(f(n))$

  Date due funzioni $f, g : \mathbb{N} \to \mathbb{R}_{\geq 0}$, diciamo che $f(n) \in \Theta(g(n))$ se per i valori costanti positivi $c_1, c_2, \overline{n}$ risulta:

  $$0 \leq c_1 g(n) \leq f(n) \leq c_2 g(n) \quad \forall n \geq \overline{n}$$

  In questo caso g descrive l'ordine di crescita preciso di f, poiché il valore di quest'ultima sarà sempre compreso fra le altre due. Formalmente:

  $$f(n) \in \Theta(g(n)) \iff f(n) \in O(g(n)) \wedge f(n) \in \Omega(g(n))$$

Esempio 1. ***Ricerca di un elemento in un array***
La complessità della ricerca di un elemento in un array è data da una funzione lineare, consentendoci di modellarla come:

$$T(n) = cn + a$$

Dove c rappresenta il costo del confronto, n è la dimensione dell'input, mentre a rappresenta i costi iniziali, espressi in una costante. Lavorando con l'analisi asintotica possiamo rimuovere i termini costanti e considerare esclusivamente il valore variabile n, ottenendo:

- *Caso ottimo:* $T(n) \in \Theta(1)$*, perché prende il primo elemento dell'array.*
- *Caso pessimo:* $T(n) \in \Theta(n)$*, perché scorre l'intero array fino all'elemento cercato.*
- *Caso medio:* $T(n) \in \Theta(n)$*, perché al netto di costanti, la complessità dipende dalla dimensione.*

Esempio 2. ***Dimostrazione di appartenenza a O(n)***
Sia la funzione $T(n) = 5n + 7 \in O(2n)$*. Dobbiamo trovare un* c *tale per cui valga l'equazione*

$$5n + 7 \leq c(2n)$$

Per far rispettare la relazione e quindi trovare un limite valido possiamo scegliere la costante $c = 3$*, con la quale, svolgendo le operazioni, otteniamo:*

$$5n + 7 \leq 6n$$

Risolvendo la disequazione infine otteniamo il valore $\overline{n} = 7$*, dimostrando la correttezza della relazione.*

Questi erano esempi concreti con funzioni ben definite, ma è possibile ragionare in termini più generali grazie alle seguenti proprietà:

- **Proprietà transitiva**

$$\begin{aligned} f(n) \in \Theta(g(n)) \wedge g(n) \in \Theta(h(n)) &\implies f(n) \in \Theta(h(n)) \\ f(n) \in O(g(n)) \wedge g(n) \in O(h(n)) &\implies f(n) \in O(h(n)) \\ f(n) \in \Omega(g(n)) \wedge g(n) \in \Omega(h(n)) &\implies f(n) \in \Omega(h(n)) \end{aligned} \tag{1.1}$$

- **Proprietà riflessiva**

$$\begin{aligned} f(n) \in \Theta(f(n)) \\ f(n) \in O(f(n)) \\ f(n) \in \Omega(f(n)) \end{aligned} \tag{1.2}$$

- **Proprietà simmetrica**

$$f(n) \in \Theta(g(n)) \iff g(n) \in \Theta(f(n))$$

- **Simmetria trasposta**

$$f(n) \in O(g(n)) \iff g(n) \in \Omega(f(n))$$

Poiché queste proprietà valgono per le notazioni asintotiche, è possibile trarre un'analogia concettuale fra il confronto asintotico di funzioni e numeri reali. In particolare:

$$\begin{aligned} f(n) \in O(g(n)) &\equiv a \leq b \\ f(n) \in \Omega(g(n)) &\equiv a \geq b \\ f(n) \in \Theta(g(n)) &\equiv a = b \end{aligned} \tag{1.3}$$

Per la dimostrazione delle relazioni nei diversi ordini di grandezza ci serviremo del metodo di **sostituzione**, con lo scopo di ricavare la tesi tramite passaggi logici. Una buona prassi è data da:

1. Scrivere i dati in base alla loro definizione.
2. Identificare ciò che è necessario per la dimostrazione.
3. Verificare la tesi.

Esempio 3. ***Dimostrazione limite asintotico superiore***
Supponiamo che la seguente formula sia vera. La si dimostri per confermare la supposizione:

$$f_1 \in O(g_1), f_2 \in O(g_2) \implies f_1 + f_2 \in O(g_1 + g_2)$$

Procediamo con i passaggi precedentemente menzionati:

1. *Riduciamo gli elementi alla loro definizione formale, ottenendo:*

$$f_1 \leq c_1 g_1(n); \quad f_2 \leq c_2 g_2(n)$$

2. *Per dimostrare la relazione è necessario trovare due valori c e $\overline{n}$ positivi. Ogni funzione ha i propri, quindi la c sarà data dalla loro somma, mentre $\overline{n}$ dal massimo di entrambi. Infatti:*

$$c = c_1 + c_2; \quad \overline{n} = max(\overline{n_1}, \overline{n_2})$$

3. *Procediamo con la verifica della tesi sostituendo agli elementi le relative definizioni:*

$$\begin{aligned} T(n) &= f_1(n) + f_2(n) \leq c_1 g_1(n) + c_2 g_2(n) \\ &= f_1(n) + f_2(n) \leq c(g_1(n) + g_2(n)) \\ &= f_1(n) + f_2(n) \leq c(g_1 + g_2)(n) \implies f_1(n) + f_2(n) \in O(g_1 + g_2) \end{aligned} \tag{1.4}$$

Naturalmente per dare una visione completa della complessità di un algoritmo bisogna considerare anche il caso ottimo, quindi il suo limite asintotico inferiore. I passaggi di dimostrazione sono fondamentalmente gli stessi di prima.

Esempio 4. ***Dimostrazione limite asintotico inferiore***
Dimostrare la seguente formula per confermare il limite inferiore dell'algoritmo:

$$f \in O(g(n)) \iff g(n) \in \Omega(f(n))$$

Stiamo asserendo che f è nell'ordine di grandezza di g se e solo se quest'ultima è il limite asintotico inferiore della prima. Procediamo:

1. *Definizione formale degli elementi:*

$$\exists c > 0, \exists \overline{n} : \forall n \geq \overline{n}, \quad f(n) \leq cg(n)$$

2. *Elementi necessari alla dimostrazione: I soliti valori $c, \overline{n}$.*

3. *Siccome la costante è positiva, effettuiamo il passaggio:*

$$cg(n) \geq f(n) \implies g(n) \geq \frac{f(n)}{c}$$

Infine diciamo che $\frac{1}{c} = c'$, che ci fa ottenere la definizione del limite inferiore:

$$g(n) \geq c'f(n) \implies g(n) \in \Omega(f(n))$$

Per riassumere, se vogliamo determinare l'ordine esatto di crescita è necessario dimostrare sia il **limite superiore** $O(f(n))$ che il **limite inferiore** $\Omega(f(n))$ rispetto alla stessa funzione, il quale è descritto da:

$$\theta(f(n)) = O(f(n)) \cap \Omega(f(n))$$

1.2 Equazioni di ricorrenza

Nella programmazione esistono due metodi principali per la costruzione di algoritmi: **ricorsione** e **iterazione**; in questa sezione studieremo entrambe le idee, con particolare attenzione agli algoritmi ricorsivi, rappresentati tramite le **equazioni di ricorrenza**. Una forma generale è data da:

$$T(n) = \begin{cases} \Theta(1) & n \leq \overline{n} \\ aT(n/b) + f(n) & n > \overline{n} \end{cases}$$

Dove, in particolare:

- **Numero dei sottoproblemi**: a.

- **Dimensione di ogni sottoproblema**: n/b.
- **Costo del lavoro svolto fuori dalle chiamate ricorsive**: $f(n)$.

Essendo che il caso base influisce esclusivamente su costanti additive e non sull'ordine asintotico, possiamo considerare solo il passo ricorsivo. Per risolvere queste equazioni ci serviremo del metodo di **sostituzione**, dove si suppone una stima asintotica della funzione, per poi provare a dimostrarla tramite induzione matematica, e degli **alberi di ricorrenza**, i quali la rappresentano con dei nodi la cui somma nel singolo livello è il costo della ricorsione in quel passo.

Esempio 5. ***Metodo di sostituzione***
Supponiamo l'equazione di ricorrenza $T(n) = 2T(\lfloor n/2 \rfloor) + \Theta(n)$. Vogliamo dimostrare che il suo limite superiore è:

$$T(n) \in O(nlogn)$$

Assumiamo $T(n) \leq c(nlogn)$ per ogni $n \geq \overline{n}$ come ipotesi induttiva per trovare i vincoli da rispettare. Stabilire la verità di questa ipotesi è sufficiente a dimostrare la tesi.

Se il limite vale per $n \geq \overline{n}$, allora l'ipotesi varrà per $n = \lfloor n/2 \rfloor$, da cui, per definizione, possiamo dire:

$$T(n) \leq c(nlogn) \implies T(\lfloor n/2 \rfloor) \leq 2(c \cdot \lfloor n/2 \rfloor log(\lfloor n/2 \rfloor) + \Theta(n)$$

Da questa forma possiamo procedere con la risoluzione dell'equazione di ricorrenza:

$$\begin{aligned} T(n) &\leq 2(c \cdot \lfloor n/2 \rfloor log(\lfloor n/2 \rfloor) + \Theta(n) \\ &\leq c \cdot nlog(n/2) + \Theta(n) \\ &\leq c \cdot nlog(n) - c \cdot nlog(2) + \Theta(n) \\ &\leq c \cdot nlog(n) - cn + \Theta(n) \end{aligned} \tag{1.5}$$

Nell'ultimo passaggio abbiamo rimosso le costanti moltiplicative. Logicamente, questo è un valore minore dell'ipotesi, quindi possiamo dimostrare la veridicità della tesi affermando che:

$$c(nlogn) \geq cnlog(n) - cn + \Theta(n)$$

Esempio 6. ***Albero di ricorsione***
Supponiamo l'equazione di ricorrenza $T(n) = 3T(n/4) + \Theta(n^2)$, definiamo il suo albero di ricorsione. Ad una prima occhiata è evidente che abbiamo 3 sottoproblemi e 4 come fattore di riduzione. Ragioniamo quindi per livelli:

- ***Livello 0 - Radice***

  Nodi: 1 *Costo:* n^2

- ***Livello 1***

  Nodi: 3 *Dimensione nodo:* $n/4$ *Costo nodo:* $(n/4)^2$ *Costo totale:* $\frac{3}{16}n^2$

- ***Livello i***

  Nodi: 3^i *Dimensione nodo:* $n/4^i$ *Costo nodo:* $(n/4^i)^2$ *Costo totale:* $\left(\frac{3}{16}\right)^i n^2$

Infine determiniamo ***altezza*** *dell'albero e* ***costo totale****. La prima è legata al totale di livelli della ricorsione. Siccome questa termina quando la somma dei nodi è uguale a 1, la ricaviamo con:*

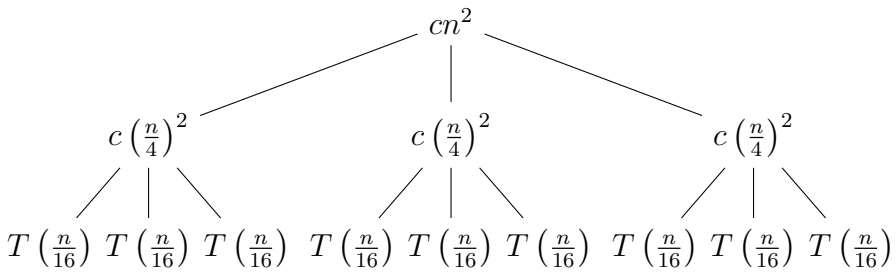
$$n/4^h = 1 \implies 4^h = n \implies h = log_4(n)$$

Il costo totale si ottiene invece facendo la somma di tutti i nodi di ogni livello, quindi si descrive con una sommatoria che va da 0 *al valore dell'altezza* $log_4(n)$*, dunque:*

$$\sum_{i=0}^{log_4(n)} cn^2 \left(\frac{3}{16}\right)^i$$

Questa è allo stesso tempo una serie geometrica con ragione $3/16 < 1$*. Questa converge, quindi il costo è dato dal primo termine:*

$$T(n) \in \Theta(n^2)$$



Per quanto riguarda la **modalità iterativa**, usiamo la notazione $f^i(n)$ per denotare la funzione f applicata iterativamente i volte a un valore di entrata n. Formalmente, sia una funzione $f(n)$ definita sui reali; per ogni intero non negativo i definiamo ricorsivamente:

$$f^i(n) = \begin{cases} n & i = 0 \\ f(f^{i-1}(n)) & i > 0 \end{cases}$$

Semplicemente, $f(n) = 2n \implies f^i(n) = 2^i n$. Un esempio più complesso è il seguente.

Esempio 7. ***Metodo iterativo***
Sia la funzione $T(n) = 1 + T(n-1)$. Per dimostrare $T(n) \in \Theta(n)$ è necessario esaurire tutte le iterazioni richieste.

$$\begin{aligned} T(n) &= 1 + T(n-1) \\ &= 1 + 1 + T(n-2) \\ &= 1 + 1 + 1 + T(n-3) \\ &= 1 + ... + 1 + T(n-i) \\ &= 1 + ... + 1 + T(n-n) \equiv 1 + ... + 1 + T(1) \end{aligned} \tag{1.6}$$

Otteniamo dunque $T(n) = n \implies T(n) \in \Theta(n)$

1.3 Teorema dell'esperto

Il **Teorema dell'esperto**, master theorem o metodo principale, è un enunciato utile per imporre una dominazione stretta su una funzione $f(n)$ tramite un polinomio di mezzo, definito come n^ϵ. Pone una condizione sufficiente ma non necessaria per le dimostrazioni e consente di risolvere più facilmente le ricorrenze in forma:

$$T(n) = aT(n/b) + f(n)$$

Poniamo una **funzione spartiacque** $n^{log_b a}$, usata per effettuare un confronto. Se una funzione è dominata da un'altra che sta sotto alla spartiacque, avremo la garanzia che la prima si troverà nell'ordine di grandezza di quest'ultima. Il teorema si definisce formalmente come:

Teorema 1. ***Teorema dell'esperto***
Siano $a > 0, b > 1$ delle costanti e $f(n)$ una funzione forzante definita e non negativa per tutti i reali maggiori di una certa soglia. Sia inoltre $T(n)$ la ricorrenza definita per $n \in \mathbb{N}$ da:

$$T(n) = aT(n/b) + f(n)$$

Dove l'espressione $aT(n/b) = a'T(\lfloor n/b \rfloor) + a''T(\lceil n/b \rceil)$ per opportune costanti $a', a'' \geq 0 : a = a' + a''$. Allora il comportamento asintotico della ricorrenza può essere caratterizzato nel seguente modo:

1. *Se esiste una costante $\epsilon > 0 : f(n) \in O(n^{log_b(a-\epsilon)})$, allora $T(n) \in \Theta(n^{log_b a})$.*
2. *Se esiste una costante $k \geq 0 : f(n) \in \Theta(n^{log_b a} \cdot log^k n)$, allora $T(n) \in \Theta(n^{log_b a} \cdot log^{k+1} n)$.*

3. *Se esiste una costante $\epsilon > 0 : f(n) \in \Omega(n^{log_b(a+\epsilon)})$ e inoltre $f(n)$ soddisfa la seguente* ***condizione di regolarità****:*

$$af(n/b) \leq cf(n) \forall n \text{ abbastanza grandi e una } c < 1$$

Allora $T(n) \in \Theta(f(n))$.

Il teorema, tuttavia, non si può usare in ogni istanza. I casi da evitare sono:

- La ricorrenza non è della forma $aT(n/b) + f(n)$.
- C'è una divisione irregolare, come $T(n-1)$.
- $f(n)$ non è confrontabile con un polinomio.
- Manca la condizione di regolarità nel caso 3.

Esempio 8. ***Caso 1 del teorema***
Sia $T(n) = 9T(n/3) + n$, abbiamo $a = 9, b = 3 \implies n^{log_3 9} = n^2$. Abbiamo inoltre $f(n) = n$; confrontandola con l'ϵ risulta:

$$n \in O(n^{2-\epsilon})$$

Ci troviamo quindi nel primo caso e confermiamo che $T(n) \in \Theta(n^2)$.

Esempio 9. ***Caso 2 del teorema***
Sia $T(n) = T(2/3n) + 1$, abbiamo $a = 1, b = 3/2, f(n) = 1$, quindi:

$$n^{log_b a} = n^{log_{3/2} 1} = n^0 = 1$$

Siccome $f(n) = 1 \implies f(n) \in \Theta(1)$ ci troviamo nel caso 2. Ogni qualvolta si riporta il problema a costanti, ci troviamo in ordine di $log(n)$, dato che rappresenta il numero dei livelli dell'albero. Confermiamo che $T(n) \in \Theta(log(n))$.

Esempio 10. ***Caso 3 del teorema***
Sia $T(n) = 3T(n/4) + nlog(n)$, abbiamo $a = 3, b = 4, f(n) = nlog(n)$, quindi:

$$n^{log_b a} = n^{log_4 3}$$

Notiamo che $nlog(n)$ cresce più rapidamente della spartiacque $n^{log_4 3}$, dunque ci troviamo nel caso 3. Verifichiamo prima la condizione di regolarità, prendendo per esempio $c = 3/4$:

$$3(n/4)log(n/4) \leq c \cdot nlog(n)$$

La condizione è vera per n grande con $c < 1$. Dunque $T(n) = \Theta(nlog(n))$

Esempio 11. ***Teorema non applicabile***
Sia $T(n) = 2T(n/2) + nlog(n)$, abbiamo $a = 2, b = 2, f(n) = nlog(n)$, quindi:

$$n^{log_b a} = n^1 = n$$

Notiamo che non ci troviamo in nessuno dei tre casi precedenti, perché la funzione $f(n)$ cresce più rapidamente di qualunque potenza positiva, quindi il teorema dell'esperto non è applicabile.

1.4 Esercizi svolti

Fondamentalmente servono:

1. Dimostrazione limiti asintotici superiori ed inferiori
2. Equazioni di ricorrenza con sostituzione, iterazione, alberi di ricorrenza e teorema dell'esperto

Capitolo 2

Ordinamenti e selezioni

2.1 Problema dell'ordinamento

Il problema dell'ordinamento dei dati in una struttura è uno dei più comuni che si incontra nella pratica e consente di introdurre vari strumenti di analisi e tecniche di progettazione standard. Formalmente viene definito come:

- **Input**: Sequenza di n oggetti $\langle a_1, ..., a_n \rangle$ sulla quale è definita una relazione di ordine totale.
- **Output**: Permutazione $\langle a'_1, ..., a'_n \rangle$ della sequenza di input tale che $a'_1 \leq ... \leq a'_n$.

In pratica, data per esempio una sequenza di numeri in input $\langle 1, 4, 2, 7, 5, 3 \rangle$, l'algoritmo risolutivo ritornerà in output $\langle 1, 2, 3, 4, 5, 7 \rangle$. La sequenza specifica in entrata è detta **istanza** del problema e per essere elaborata è necessario che appartenga al dominio per cui l'algoritmo è definito.

Diciamo che un algoritmo è **corretto** se, per ogni istanza valida del problema, termina e produce un output conforme alla specifica e sebbene si tratti di un concetto intuitivamente semplice, esistono molti algoritmi adatti a questo compito, la cui scelta è determinata da alcuni fattori come la **dimensione** del numero di dati, il loro **ordinamento iniziale**, eventuali **vincoli** o anche limiti fisici dell'elaboratore.

2.2 Insertion Sort

Algoritmo efficiente per ordinare un piccolo numero di elementi, opera allo stesso modo in cui si ordinerebbe un mazzo di carte. Dato in input un array A, mantengo una parte ordinata ed una non ordinata; ad ogni iterazione si prende un elemento dalla seconda e lo si inserisce nella posizione corretta, trovata confrontando tutte le carte a partire dall'inizio.

- **Caso pessimo**: $\Theta(n^2)$
- **Caso ottimo**: $\Theta(n)$. Si ottiene quando l'array è già ordinato.
- **Caso medio**: $\Theta(n^2)$
- **Ordinamento in loco**: Sì
- **Stabilità**: Sì

Algorithm 1 InsertionSort(A,n)

Require: A: Array, n: Dimensione array

```
for i ← 2 to n do                    ▷ Scorre da startPos+1=2 fino a n dimensione
    key ← A[i]
    j ← i − 1                        ▷ L'indice j è usato per la sezione ordinata. Valori: [1,n]

    while j > 0 AND A[j] > key do    ▷ Non sei all'inizio e key minore di A[i]
        A[j + 1] ← A[j]              ▷ Sposta a destra di una posizione il valore maggiore di key
        j ← j − 1                    ▷ Spostati a sinistra di una posizione
    end while
    A[j + 1] ← key                   ▷ Inserisci key nella posizione corretta
end for
```

La valutazione della complessità di questo algoritmo vede analizzati due elementi principali: il ciclo **for**, avente tre istruzioni (gli assegnamenti) di complessità costante, e il ciclo **while** interno al primo, il quale, potendo scorrere potenzialmente l'intero array ad ogni iterazione, aumenta notevolmente i tempi di esecuzione.

Dove richiede un tempo di lavoro generalmente alto rispetto ad altri algoritmi, la sua potenzialità sta nel risparmio della memoria; infatti richiede solo tre variabili per essere eseguito e non richiede alcun supporto esterno per la gestione dell'array. Diciamo, per questa ragione, che l'algoritmo **opera in loco**. Un'altra sua caratteristica è che gli elementi uguali non vengono riordinati fra di loro poiché il confronto effettuato è stretto. In queste condizioni, diciamo che l'algoritmo è **stabile**.

2.3 Merge Sort

Algoritmo basato sulla tecnica del **Divide et Impera**, ovvero nella suddivisione del problema in sottoproblemi della stessa natura di dimensione via via più piccola.

Diviso l'array di partenza in due parti uguali, si ordinano entrambe applicando ricorsivamente l'algoritmo, per poi combinare queste due nuove sezioni con un sottoalgoritmo **merge** di supporto, prendendo volta per volta il valore più piccolo contenuto in entrambi, fino ad esaurire tutti i valori.

- **Caso pessimo**: $\Theta(nlog(n))$
- **Caso ottimo**: $\Theta(nlog(n))$
- **Caso medio**: $\Theta(nlog(n))$
- **Ordinamento in loco**: No, richiede memoria ausiliaria.
- **Stabilità**: Sì

Algorithm 2 merge(A, left, center, right)

Require: A: Array, left: Posizione iniziale, center: Centro, right: Posizione finale

```
i ← left
j ← center + 1
k = 1                                   ▷ Indice dell'array di supporto
B[right − left + 1]                     ▷ Array di supporto di dimensione right-left+1

while i ≤ center AND j ≤ right do       ▷ Fintanto che i sottarray non sono finiti
    if A[i] ≤ A[j] then                 ▷ Se l'elemento del primo è minore, copia quello e avanza
        B[k] ← A[i]
        i ← i + 1
        k ← k + 1
    else                                ▷ Altrimenti procedimento analogo, ma per il secondo array
        B[k] ← A[j]
        j ← j + 1
        k ← k + 1
    end if
end while
while i ≤ center do                     ▷ Copia il primo array ordinato
    B[k] ← A[i]
    i ← i + 1
    k ← k + 1
end while
while j ≤ right do                      ▷ Copia il secondo array ordinato
    B[k] ← A[j]
    j ← j + 1
    k ← k + 1
end while
for k ← left to right do                ▷ Sovrascrivi l'array di base con quello ordinato
    a[k] ← b[k − left]
end for
```

Algorithm 3 mergeSort(A, left, right)

Require: A: Array, left: Posizione iniziale, right: Posizione finale

```
if left < right then
    center = (left+right)/2
    mergeSort(A, left, center)          ▷ Ordina la parte prima della metà
    mergeSort(A, center+1, right)       ▷ Ordina la parte dopo la metà
    merge(A, left, center, right)       ▷ Fondi i due sottoarray ordinati
end if
```

Diversamente rispetto ad Insertion Sort, per valutare la complessità di questo algoritmo bisogna considerare la ricorsione. Sapendo che ad ogni passo ricorsivo il problema è diviso in **due sottoproblemi** ($n/2$), dove si effettuano **due chiamate ricorsive** ($2T$), e che il merge scorre tutti gli elementi **almeno una volta** $\Theta(n)$, possiamo dire che Merge Sort è rappresentato da:

$$T(n) = 2T(n/2) + \Theta(n)$$

Possiamo usare il teorema dell'esperto per la dimostrazione. Convertendo la ricorrenza alla forma generale, otteniamo:

$$a = 2, \quad b = 2, \quad f(n) = \Theta(n), \quad log_2(2) = 1 \implies n^1 = n$$

Confrontando $f(n)$ con la spartiacque, notiamo che appartengono allo stesso ordine di grandezza, facendoci trovare nel caso 2 del teorema, infatti:

$$f(n) = \Theta\left(n^{log_b(a)}\right) \implies T(n) = \Theta(nlog(n))$$

2.4 Struttura Heap e Heap Sort

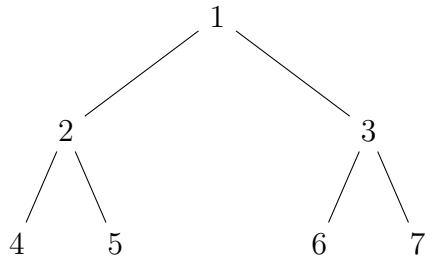
Introduciamo ora la prima struttura di dati del corso. Nelle sezioni precedenti è stato menzionato il concetto di **albero binario**, una struttura composta in cima da un nodo, detto **radice**, il quale, in quanto binario, si dirama in due direzioni verso il basso attraverso **archi**, creando due nodi figli. In particolare, quando un nodo figlio è alla base dell'albero si dice **foglia**.

Come è possibile intuire, gli alberi binari sono organizzati in livelli, i quali indicano la **profondità**, ovvero la distanza dalla radice. Quando un livello ha il numero massimo di nodi ivi ottenibili si dice **completo**. Tuttavia possono esistere alberi **semi-completi**, quando la base non è completa.

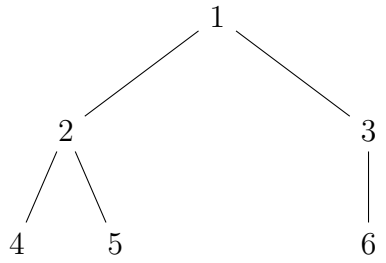
Un vantaggio di questa struttura è la semplicità della ricerca dei nodi; infatti, la dinamica si dispiega in tre casi:

- **Per figlio sinistro**: indice $\cdot\, 2 + 0$

- **Per figlio destro**: indice $\cdot\, 2 + 1$
- **Da figlio a padre**: indice/2



Albero binario completo



Albero binario semi-completo

Un **heap** è una struttura dati implementata tramite un array, considerata sotto forma di un albero binario semi-completo, i cui nodi rappresentano gli elementi al suo interno. Deve inoltre rispettare la proprietà che lo contraddistingue: in un **MAX-Heap** ogni nodo è maggiore o uguale ai propri figli. Analogamente, è possibile definire un **MIN-Heap** dove vale il contrario. Nella gestione di questa struttura dati, tuttavia, in necessità di inserimento nodi, non necessariamente si potrà rispettare sempre la proprietà, ragion per cui utilizziamo un algoritmo **heapify()** ad-hoc, con lo scopo di prendere questo problema locale e trasferirlo alle foglie, dove sarà risolto.

Algorithm 4 heapify(A, i)

Require: A: Array, i: Indice

```
l ← left(i), r ← right(i)                    ▷ l prende figlio-sx, r prende figlio-dx

if l ≤ A.heapSize AND A[l] > A[i] then       ▷ Se figlio-sx ha valore > i, non è heap.
    largest ← l
else
    largest ← i                              ▷ Altrimenti segna che il più grande è i, come inteso.
end if

if r ≤ A.heapSize AND A[r] > A[largest] then ▷ Controllo figlio destro
    largest ← r
end if

if largest ≠ i then                          ▷ Se la proprietà non è valida
    switch(A[i], A[largest])                 ▷ Scambia la posizione dei due nodi, largest è l'anomalia
    heapify(A, largest)                      ▷ Sposta ricorsivamente largest finché non rispetta la regola
end if
```

La complessità di questo algoritmo è equivalente alla sua profondità, quindi il caso peggiore scende lungo l'altezza dell'albero fino alle foglie, dando come tempo di esecuzione $T(n) \in O(log(n))$.

2.5 Quick Sort classico e probabilistico

2.6 Counting Sort

2.7 Radix Sort

2.8 Bucket Sort

2.9 Problema della selezione

Capitolo 3

Strutture dati

3.1 Heap

3.2 Alberi binari

3.2.1 RB-alberi

3.2.2 B-alberi binomiali

3.3 Tabelle hash

3.4 Code con priorità

3.5 Insiemi disgiunti

3.6 Estensione di strutture dati

3.7 Grafi

Capitolo 4

Progetto e analisi di algoritmi

4.1 Divide et impera

4.2 Programmazione greedy

4.3 Programmazione dinamica

4.4 Ricerca locale

4.5 Backtracking

4.6 Branch and bound

Capitolo 5

Algoritmi fondamentali

5.1 Alberi di copertura di costo minimo

Prim, Kruskal

5.2 Programmazione lineare

Simplesso, algoritmo fondamentale polinomiale basato sugli ellissoidi

5.3 Cammini minimi

5.3.1 Sorgente singola

Dijkstra, Bellman-Ford

5.3.2 Sorgente multipla

Floyd-Warshall, Johnson

5.4 Flusso massimo

Ford-Fulkerson, Karp

5.5 Matching massimale su grafo bipartito